

Towards Practical On-Device Android Malware Detection via Lightweight Static Analysis

Abdullah Faiyaz

Department of CSE, BUET

2005065@ugrad.cse.buet.ac.bd

Jarin Tasnim

Department of CSE, BUET

2005083@ugrad.cse.buet.ac.bd

Sadnam Faiyaz

Department of CSE, BUET

2005111@ugrad.cse.buet.ac.bd

Md. Mostofa Akbar

Department of CSE, BUET

mostofa@cse.buet.ac.bd

Abstract—Detection of malware on Android has come a long way, with both static and dynamic analysis methods. But the question of implementing them in real-life, constrained environments with limited battery, CPU, and memory still remains answered by very few approaches. Taking the question seriously, we present **VigiDroid**, which is a fully offline static analysis framework that runs entirely on the device itself. We have experimented with different feature sets (manifest attributes, API calls, DEX header metadata, and raw APK byte sequences), pairing each with lightweight classifiers such as XGBoost, multilayer perceptrons, and 1D CNNs. Every model is exported to ONNX and executed on-device via ONNX Runtime, enabling direct evaluation on Android devices. In addition to accuracy and F1-score, we profile feature extraction, vectorization, and inference times, CPU utilization, and memory consumption. Our preliminary results show that; mention experimental best result + resource usage tradeoff.

Index Terms—Android malware detection, on-device inference, static analysis, multi-DEX, XGBoost, 1D-CNN, ONNX Runtime

I. INTRODUCTION

Over the years, researchers have made real progress in detecting Android malware — static analysis, dynamic analysis, and hybrid approaches have all shown impressive results on paper. But the gap that rarely gets addressed is that most of these techniques were designed and evaluated on powerful machines, not on the phones people actually carry around with limited battery, CPU, and memory. In this paper, we introduce **VigiDroid**, a fully offline static analysis framework that runs entirely on the device itself. We experimented with different feature sets (manifest attributes, API calls, DEX header metadata, and even raw APK byte sequences) paired with lightweight models such as XGBoost, multilayer perceptrons, and 1D CNNs. All models are deployed using ONNX Runtime and evaluated directly on Android devices. In addition to detection accuracy and F1-score, we profile feature extraction, vectorization and inference time, CPU utilization, and memory consumption. Preliminary experiments show that; mention experimental best result + resource usage tradeoff.

II. BACKGROUND & RELATED WORKS

Android malware detection has been studied extensively using both static and dynamic analysis. Like permission-based linear models [2], broadcast-receiver mining [4], permission extraction frameworks (MLDP) [8], byte-level 1-D CNNs on raw APK tails [1], and structural DEX/manifest fusion architectures inspired by MSFDroid-style [11] multi-branch

designs. Early efforts like SeqMobile [5] showed that on-device detection is possible by using RNN-based models on permissions, intents, and API call sequences. SAMADroid [6] took a hybrid approach, combining lightweight on-device checks with cloud-based dynamic analysis. However, its reliance on certain system privileges has become problematic as modern Android versions have tightened security restrictions. DL-Droid [7] ran dynamic analysis directly on real devices, but it required installing and fully executing each app which make it impractical for everyday scanning. Overall, these works highlight that building a malware detector that is both accurate and lightweight enough to run comfortably on a real phone remains an unsolved problem, especially when balancing strong detection performance against tight mobile resource constraints.

III. PROPOSED METHODOLOGY

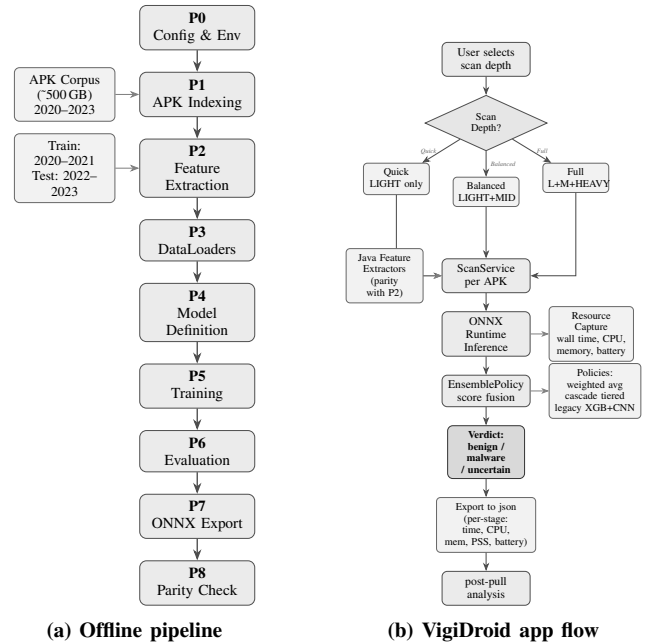


Fig. 1. Proposed VigiDroid methodology. *Left*: the offline model-development pipeline, with export and model-family details omitted for compactness. *Right*: the on-device scan pipeline that selects a scan depth, extracts features locally, runs ONNX inference, fuses scores, and exports resource metrics.

TABLE I

EXPERIMENTAL COMPARISON OF ON-DEVICE MALWARE DETECTION MODELS. OFFLINE METRICS ARE COMPUTED ON THE COMPUTER TEMPORAL TEST SPLIT; RESOURCE COLUMNS ARE DERIVED FROM ON-DEVICE ALL_SCAN_METRICS.JSON.

Model	Features	Acc.	F1	ROC-AUC	CPU	Mem.	Batt.	Time	Feasible
XGBoost	Manifest perms, intents, APIs	0.9372	0.8884	0.9913	97.4 ms	0.02 MB	477.102	100.0 ms	low
1D-CNN	Last 1024 APK bytes	0.9005	0.8486	0.9592	1.7 ms	0.01 MB	8.433	1.7 ms	low
MLP-H	Dex header, sum-pooled	0.9686	0.9478	0.9675	0.7 ms	0.01 MB	1.997	0.4 ms	high
Dex+Manifest ASCNN	Dex header + manifest BoW	0.9411	0.9064	0.9685	7.8 ms	0.03 MB	36.045	7.9 ms	medium
Dex+Manifest Dual	Dex header + manifest, dual-branch	0.9686	0.9483	0.9805	7.9 ms	0.02 MB	36.84	7.9 ms	medium
LinRegDroid	Manifest permission flags	0.9175	0.8645	0.9259	1.0 ms	0.00 MB	3.008	0.7 ms	low
MLDP-pruned	MLDP-selected permissions	0.9319	0.8855	0.9017	0.6 ms	0.00 MB	1.61	0.4 ms	medium
Broadcast+MLDP	MLDP perms + receiver actions	0.9359	0.8928	0.9373	1.1 ms	0.00 MB	1.76	0.4 ms	medium
MLDP+Dex Cascade	MLDP perms + dex header	0.9581	0.9298	0.9469	0.8 ms	0.01 MB	4.919	0.4 ms	high
Dex+Broadcast Fusion	Dex header + receiver actions	0.9699	0.9503	0.9794	0.9 ms	0.01 MB	2.362	0.5 ms	high

IV. EXPERIMENTS

We evaluate each model on two axes: **offline quality** (accuracy, F1, ROC-AUC on temporal holdout APKs) and **on-device feasibility** (mean CPU, memory, battery, wall time per scan on a physical Android device). Table I reports the planned combined comparison; values below are placeholders pending completed device campaigns and unified offline exports. Fig. 2 illustrates

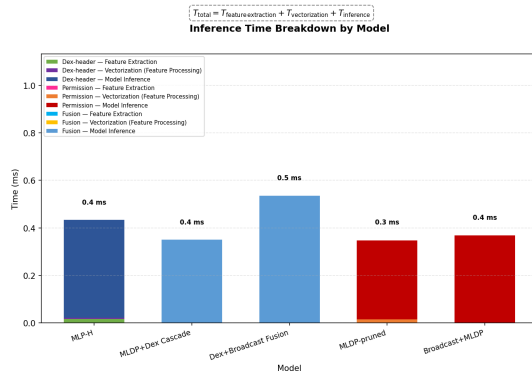


Fig. 2. Inference time vs APK size.

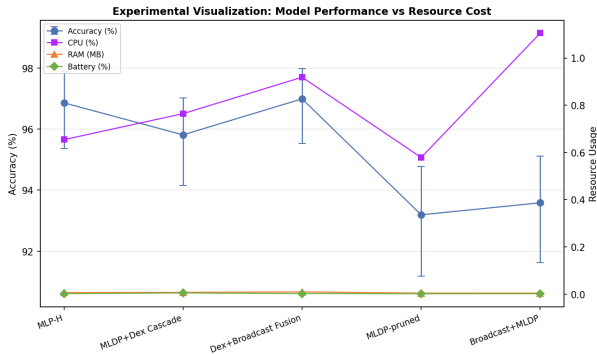


Fig. 3. Planned trade-off figure:

Protocol (planned). Labeled eval APKs (eval_XXXX_benign/malware.apk) on device; metrics pulled via ADB; offline labels from corpus year splits.

V. CONCLUSION

We described VigiDroid, a thesis framework for practical on-device Android malware pre-screening that unifies literature-based static detectors through a disciplined train-export-deploy pipeline and tiered ONNX orchestration on Android. By logging multi-dimensional resource metrics alongside fused verdicts, the system supports evidence-based selection of models that balance accuracy and mobile cost. Final quantitative results, ensemble calibration, and user-study implications will be reported once device experiments and figure generation are complete.

ACKNOWLEDGMENT

The authors thank their supervisor and collaborators at the Department of Computer Science and Engineering, BUET, for guidance throughout this thesis.

REFERENCES

- [1] C. Hasegawa and H. Iyatomi, "One-dimensional convolutional neural networks for Android malware detection," in *Proc. ICCE*, 2020.
- [2] A. Narayanan et al., "LinRegDroid: Detection of Android malware using multiple linear regression," *Computers & Security*, 2018.
- [3] Tao Peng et al., "A Lightweight Multi-Source Fast Android Malware Detection Model," *Appl. Sci.*, 2022.
- [4] M. Mohsen and A. Ismail, "Detecting Android malwares by mining statically registered broadcast receivers," *Int. J. Advanced Computer Science and Applications*, 2017.
- [5] Ruitao Feng et al., "SeqMobile: An Efficient Sequence-Based Malware Detection System Using RNN on Mobile Devices" *25th International Conference on Engineering of Complex Computer Systems (ICECCS)*, 2020.
- [6] Saba Arshad et al., "SAMADroid: A Novel 3-Level Hybrid Malware Detection Model for Android Operating System" *IEEE Access*, 2018.
- [7] Mohammed K. Alzaylaee et al., "DL-Droid: Deep learning based android malware detection using real devices" *IEEE Access*, 2020.
- [8] S. Wang et al., "Permission extraction framework for Android malware detection," *IEEE Access*, 2019.
- [9] Y. Li et al., "MSFDroid: Multi-stage fusion for Android malware detection," *IEEE Trans. Information Forensics and Security*, 2022.
- [10] Bragança, H., Kreutz, D., Rocha, V. et al., "MH-1M: A 1.34 Million-Sample Multi-Feature Android Malware Dataset with Rich Metadata," *Sci Data* 13, 153. <https://doi.org/10.1038/s41597-025-06469-5>, 2026.
- [11] Tao Peng et al., "A Lightweight Multi-Source Fast Android Malware Detection Model," *Appl. Sci.*, 2022.
- [12] D. Arp et al., "Drebin: Effective and explainable detection of Android malware in your pocket," in *Proc. NDSS*, 2014.
- [13] ONNX Runtime documentation, <https://onnxruntime.ai/>, accessed 2026.